

Date:

Exp No:4

Aim: Inter-process communication between pipes , message queues and shared memory

1.Pipes

Using Pipe—Evaluate the expression. The first process evaluates. The second process evaluates and send it to the first process which evaluates the final expression and displays it.

Theory

A pipe is an Inter-Process Communication (IPC) mechanism that allows one process to send data to another process in a unidirectional manner. It provides a way for processes to communicate using a temporary memory buffer managed by the operating system.

Working of Pipes

A pipe has two ends:

- Write end – used to send data
- Read end – used to receive data

Data written at the write end of the pipe is read from the read end by another process.

Pipes are mainly used for communication between related processes, such as a parent and its child process.

1.Write the c program OS_exp4.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
    int fd[2];
    int a = 10, b = 20;
    int c = 5, d = 5;
    int part1, part2, result;

    pipe(fd);

    if (fork() == 0) {
        // Child process
        close(fd[0]);           // close read end
        part2 = c + d;          // evaluate second part
        write(fd[1], &part2, sizeof(part2));
        close(fd[1]);
    }
    else {
        // Parent process
        close(fd[1]);           // close write end
        part1 = a + b;          // evaluate first part
        read(fd[0], &part2, sizeof(part2));
        result = part1 * part2;  // final evaluation
        printf("Final Result = %d\n", result);
        close(fd[0]);
    }
}
```

```
}
```

```
return 0;
```

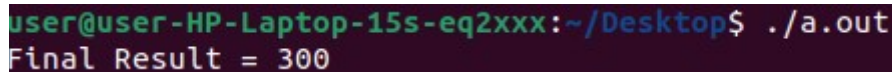
```
}
```

2.Compile and Run

```
gcc OS_exp4.c
```

```
./a.out
```

3. Expected output



```
user@user-HP-Laptop-15s-eq2xxx:~/Desktop$ ./a.out
Final Result = 300
```

ii)Message Queue

The first process sends a string to the second process.

The second process reverses the received string and sends it back to the first process. The first process compares the original string and the reversed string received from the second one and then prints whether the string is a palindrome or not.

Theory

A message queue is an Inter-Process Communication (IPC) mechanism that allows processes to send and receive messages in a structured format. Unlike pipes, message queues allow processes to communicate without requiring a direct relationship (parent-child).

Messages are stored in a kernel-managed queue until they are read by the receiving process.

Working of Message Queues

Each message queue is identified by a unique key.

A process can send a message to the queue, and another process can receive it later.

Each message has:

- Message type
- Message data

The receiving process can choose which message to read using the message type.

1.Write the c program OS_exp4_1.c

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/msg.h>
```

```
#include <unistd.h>
```

```
struct msg {
```

```
    long type;
```

```
    char text[100];
```

```
};
```

```

int main() {
    int msgid = msgget(1234, 0666 | IPC_CREAT);
    struct msg m;
    char original[100], reversed[100];
    int i, len;

    if (fork() == 0) {
        // CHILD PROCESS

        msgrcv(msgid, &m, sizeof(m.text), 1, 0);
        strcpy(original, m.text);

        // reverse string
        len = strlen(original);
        for (i = 0; i < len; i++)
            reversed[i] = original[len - i - 1];
        reversed[len] = '\0';

        m.type = 2;
        strcpy(m.text, reversed);
        msgsnd(msgid, &m, sizeof(m.text), 0);
    }
    else {
        // PARENT PROCESS

        printf("Enter a string: ");
        scanf("%s", original);

        m.type = 1;
        strcpy(m.text, original);
        msgsnd(msgid, &m, sizeof(m.text), 0);

        msgrcv(msgid, &m, sizeof(m.text), 2, 0);
        strcpy(reversed, m.text);

        if (strcmp(original, reversed) == 0)
            printf("String is Palindrome\n");
        else
            printf("String is Not Palindrome\n");

        msgctl(msgid, IPC_RMID, NULL);
    }

    return 0;
}

```

2.Compile and Run

```

gcc OS_exp4_1.c
./a.out

```

3.Expected Output

```
Enter a string: malayalam
String is Palindrome
user@user-HP-Laptop-15s-eq2xxx:~/Desktop$ ./a.out
Enter a string: english
String is Not Palindrome
user@user-HP-Laptop-15s-eq2xxx:~/Desktop$
```

3.SharedMemory

Shared memory is an Inter-Process Communication (IPC) mechanism that allows multiple processes to share a common region of memory. Processes can read and write to this shared area to exchange data efficiently.

Working of Shared Memory

A shared memory segment is created by one process and is mapped into the address space of other processes. All attached processes can access the shared region as if it were their own memory.

Once communication is complete, the shared memory segment must be detached and removed from the system.

It is the fastest IPC mechanism, since data is not copied between processes but accessed directly from memory.

1.Write the c program OS_exp4_2.c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>

int main() {
    int shmid = shmget(1234, 1024, 0666 | IPC_CREAT);
    char *shm = shmat(shmid, NULL, 0);

    char s1[50], s2[50], s3[50];
    int i;

    if (fork() == 0) {
        // CHILD PROCESS

        sscanf(shm, "%s %s %s", s1, s2, s3);

        sprintf(shm, "%s %s %s", s1, s2, s3);
```

```

}
else {
    // PARENT PROCESS

    printf("Enter three strings:\n");
    scanf("%s %s %s", s1, s2, s3);

    sprintf(shm, "%s %s %s", s1, s2, s3);

    sleep(1); // wait for child to concatenate

    // flip case
    for (i = 0; shm[i] != '\0'; i++) {
        if (islower(shm[i]))
            shm[i] = toupper(shm[i]);
        else if (isupper(shm[i]))
            shm[i] = tolower(shm[i]);
    }

    printf("Final Output: %s\n", shm);

    shmdt(shm);
    shmctl(shmid, IPC_RMID, NULL);
}

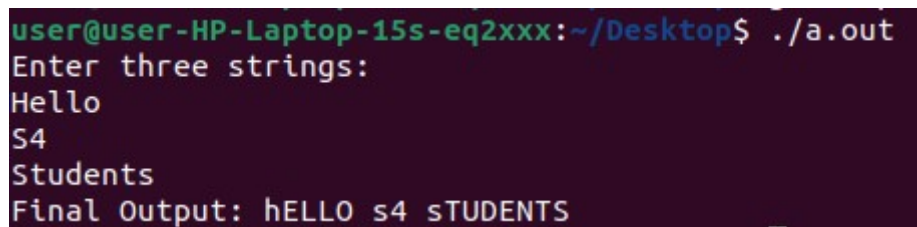
return 0;
}

```

2.Compile and Run

```
gcc OS_exp4_2.c
./a.out
```

3.Expected Output



```

user@user-HP-Laptop-15s-eq2xxx:~/Desktop$ ./a.out
Enter three strings:
Hello
S4
Students
Final Output: hELLO s4 sTUDENTS

```

Result

The program for Inter-Process Communication (IPC) using Pipes, Message Queues, and Shared Memory was successfully executed.